

Problem Set 9 - Functions and recursion

Hello student,

the goal of this problem-set is to work with functions, and tip your toes in the use of recursion.

Recursion is a powerful tool to solve a complex problem using a simple algorithm.

Assignment 9.1 - Recursive Pi (optional)

In assignment 5.2 you were asked to develop a program capable of approximate π using iterations. In this exercise you are asked to do the same, but this time using recursion.

Ask the user for the number of iterations and calculate its value.

The output should look like this (in read the input inserted by the user):

```
Insert the number of iterations: 1
After 1 iterations Pi: 4
```

```
Insert the number of iterations: 42
After 42 iterations Pi: 3.117786501758878
```

Hint: The formula to approximate π is: $\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \dots$

Hint 2: Remember if you get a `RecursionError: maximum recursion depth exceeded in comparison` you can increase (reasonably) your limit with:

```
import sys
sys.setrecursionlimit(10000)
```

Assignment 9.2 - Fees calculator (optional)

You are a tax officer at the Swiss Post and you are still calculating taxes by hand. However, you are a smart tax officer, and you want to create a small program that will help you in your calculations.

You have the approval from your boss Ruth Olga Bucher, but she is interested and prepared a main function for you. Good girl!

However, she does not know how to do the calculation. She was able to find the explanations on the [Swiss Post's website](#) where you read:

The customs clearance fee depends on the origin of the consignment and the value of the goods sent. For consignments from Germany, France, Austria and Italy, the basic fee is CHF 11.50 while for consignments from any other country, the basic fee is CHF 16.00. A surcharge corresponding to 3% of the declared value of the goods is then applied.

Examples:

A parcel from France with a value of CHF 100.00:

Basic fee: CHF 11.50 Surcharge: CHF 3.00 (3% of the declared value of the goods) Total customs clearance fee: CHF 14.50

A parcel from the US with a value of CHF 300.00:

Basic fee: CHF 16.00 Surcharge: CHF 9.00 (3% of the declared value of the goods) Total customs clearance fee: CHF 25.00

Some orders have a special value multiplier to be used instead of the normal 3%. You have freedom to write the function to calculate the fees, however the main is defined by your boss, and is the following:

```
orders = [
    {'order_id': 164355, 'origin': 'China', 'value': 119, 'multiplier': 1},
    {'order_id': 317379, 'origin': 'Germany', 'value': 455, 'multiplier': 17},
    {'order_id': 457635, 'origin': 'Italy', 'value': 412, 'multiplier': 1},
    {'order_id': 520675, 'origin': 'USA', 'value': 368, 'multiplier': 1},
    {'order_id': 605955, 'origin': 'Germany', 'value': 303, 'multiplier': 1},
    {'order_id': 762339, 'origin': 'China', 'value': 364, 'multiplier': 17},
    {'order_id': 935459, 'origin': 'Portugal', 'value': 235, 'multiplier': 1},
    {'order_id': 897891, 'origin': 'Russia', 'value': 204, 'multiplier': 1},
    {'order_id': 150915, 'origin': 'Russia', 'value': 396, 'multiplier': 1},
    {'order_id': 249571, 'origin': 'China', 'value': 481, 'multiplier': 1},
    {'order_id': 302595, 'origin': 'Spain', 'value': 320, 'multiplier': 1},
    {'order_id': 326723, 'origin': 'China', 'value': 496, 'multiplier': 1},
    {'order_id': 492515, 'origin': 'USA', 'value': 325, 'multiplier': 1},
    {'order_id': 581827, 'origin': 'Spain', 'value': 187, 'multiplier': 1},
    {'order_id': 703939, 'origin': 'Italy', 'value': 311, 'multiplier': 18},
    {'order_id': 965699, 'origin': 'Spain', 'value': 394, 'multiplier': 1},
    {'order_id': 258979, 'origin': 'Spain', 'value': 153, 'multiplier': 1},
    {'order_id': 154883, 'origin': 'USA', 'value': 362, 'multiplier': 1},
    {'order_id': 310659, 'origin': 'China', 'value': 312, 'multiplier': 1},
    {'order_id': 645539, 'origin': 'Spain', 'value': 413, 'multiplier': 1},
]

if __name__ == "__main__":
    for order in orders:
        if order["multiplier"] != 1:
            order["fees"] = calculate_fees(origin=order["origin"], value=order
                ["value"], taxes=order["multiplier"])
            continue
        order["fees"] = calculate_fees(origin=order["origin"], value=order["
            value"])

    pretty_print(orders)
```

The result will be the following (example with first 5 elements):

```
order 164355, fees: 19.57
order 317379, fees: 88.85
order 457635, fees: 23.86
order 520675, fees: 27.04
order 605955, fees: 20.59
...
```

Remember to **round results** to 2 digits!

For this exercise you don't need recursion.

Assignment 9.3 - Digits sum (optional)

We want to implement a function that is able to sum the number of digits of a certain number. For example, the sum of the digits of the number 98345 is $9 + 8 + 3 + 4 + 5 = 29$.

Create a first implementation using **iteration**.

Create then another version of the function that works **recursively**.

Use implemented functions in your main to check they work properly.

Assignment 9.4 - Recursive prime numbers (optional)

In the last problem set we developed an iterative version of a function to calculate prime numbers. This time we want to do the same but creating a **recursive** implementation.

Use then the implemented function in your main to discover and print on the screen all the numbers that are prime between 1 and 100.

Assignment 9.5 - Fibonacci - and the importance of caching (optional)

The Fibonacci numbers is a numeric sequence described as:

$$F_n = F_{n-1} + F_{n-2}$$

Starting with $F_1 = F_2 = 1$, and $F_0 = 0$.

Write a `fibonacci` function that calculates the n^{th} element in the sequence using **recursion**.

For example, using this main:

```
def fibonacci(n):  
    ... # your implementation  
  
if __name__ == "__main__":  
  
    for i in range(5):  
        print(fibonacci(i))
```

The result obtained is:

```
0  
1  
1  
2  
3
```

Now try to measure the execution time of your function using the following code:

```
import time

def fibonacci(n):
    ... # your implementation

def measure_execution_time(algorithm, number):
    tic = time.perf_counter()
    algorithm(number)
    toc = time.perf_counter()

    print(f"Element calculated in {toc - tic:0.5f} seconds")

if __name__ == "__main__":
    measure_execution_time(fibonacci, 20)
```

Now try to develop a second version of the algorithm. It works just the same but you use a dictionary as a cache.

If a number is not in the dictionary, you calculate it and store the value in the dictionary using the position (n) as key.

Obtaining something similar to:

```
import time

def measure_execution_time(algorithm, number):
    tic = time.perf_counter()
    algorithm(number)
    toc = time.perf_counter()

    print(f"Element calculated in {toc - tic:0.5f} seconds")

def fibonacci(n):
    ... # your implementation

cache = {}

def fast_fibonacci(n):
    if n in cache.keys():
        return cache[n]

    ... # something

    cache[n] = ... # something else
    return cache[n]

if __name__ == "__main__":
    measure_execution_time(fibonacci, 20)
    measure_execution_time(fast_fibonacci, 200)
```

How do the two compare to each other?